App Development Project Report

App Dev Project Report

1. Student Details

Name: Anirudh Kamalakannan
Roll Number: 24f2003534
Email: 24f2003534@ds.study.iitm.ac.in
About Me: I am an engineering student who is trying to diversify his expertise into other fields such as app development.

---

2. Project Details

Project Title: Placement Portal

Problem Statement:

The application was built as a role-based recruitment platform using Flask for the backend API and VueJS for the modular frontend interface. It enables secure, tier-based workflows for administrators, companies, and students, leveraging Redis caching for optimized searches and Celery task queues to handle asynchronous reporting and automated notifications

---

3. AI/LLM Declaration

I used **Gemini** to assist in writing the vue templates. I used some code assistance to write the backend. The extent of AI/LLM usage is around **20-30%**, limited to **code suggestions and vue templates and some debugging.**.

---

4. Technologies and Frameworks Used

Technology / Library	Purpose

Flask	Core backend web framework
Flask-SQLAlchemy	Object Relational Mapper (ORM) for the SQLite database
Vue 3	Frontend JavaScript framework for building dynamic and interactive user interfaces
Vue3-SFC-Loader	Loading Vue Single File Components (`.vue`) directly in the browser without a build step
Bootstrap 5	Frontend styling, UI components, and responsive design
Celery	Asynchronous task queue for background processes (e.g., sending daily email reminders, exporting applications)
Redis	In-memory data structure store, used as the message broker for Celery and server-side caching (`Flask-Caching`)
PyJWT	User authentication and generating JSON Web Tokens (JWT) for secure API endpoints

---

5. Database Schema / ER Diagram

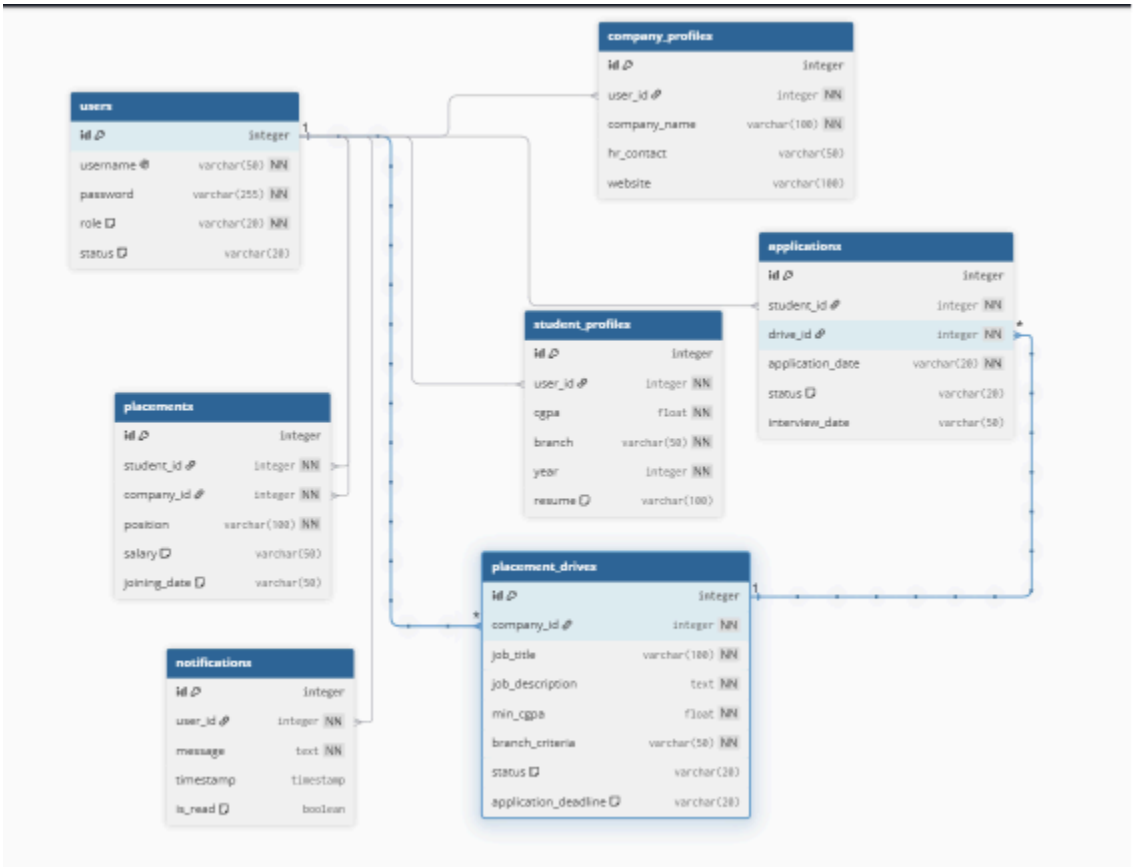
ables:

- ***users:*** *Stores application authentication details and master role profiles (id, username, password, role, status).*
- ***company_profiles:*** *Tracks organizational data and corporate representative contacts (id, user_id, company_name, hr_contact, website).*
- ***student_profiles:*** *Contains individual academic criteria, course streams, and hosted resumes (id, user_id, cgpa, branch, year, resume).*
- ***placement_drives:*** *Manages company recruitment events, tracking role details, eligibility criteria, and process active states (id, company_id, job_title, job_description, min_cgpa, branch_criteria, status, application_deadline).*
- ***applications:*** *Records student submission paths to target placement drives, managing continuous selection lifecycle metrics (id, student_id, drive_id, application_date, status, interview_date).*
- ***placements:*** *Documents official hiring confirmation contracts mapping successful candidates to hiring firms (id, student_id, company_id, position, salary, joining_date).*

- ***notifications:*** *Stores live structural message updates broadcast to users across event lifecycles (id, user_id, message, timestamp, is_read).*

Relationships:

- ***One-to-One / One-to-Many Extensions:***
  - *users --> company_profiles (One user account maps to their unique corporate profile details)*
  - *users --> student_profiles (One user account maps to their unique candidate profile credentials)*
- ***One-to-Many:***
  - *users (Company) --> placement_drives (One company user hosts multiple recruitment event postings)*
  - *users (Student) --> applications (One student account generates multiple job applications over time)*
  - *placement_drives --> applications (One job posting collects multiple application records from various candidates)*
  - *users --> notifications (One system user receives multiple time-stamped status alerts)*
- ***Placement Tracking Mappings:***
  - *users (Student) --> placements (One student can be logged under official placement fulfillment items)*
  - *users (Company) --> placements (One company can hire multiple different placement candidates)*



6.

API Resource EndpointsPOST /api/auth/register — Register a new user profile (Company or Student).

POST /api/auth/login — Authenticate user and generate a JWT session token.

GET /api/admin/stats — Fetch overall platform statistics for the admin dashboard.

GET /api/admin/users — List and search all non-admin users (companies and students).

POST /api/admin/users/<id>/status — Update a user's account status (e.g., Approve, Blacklist).

GET /api/admin/drives — Fetch all placement drives across all companies.

POST /api/admin/drives/<id>/status — Approve or reject a specific placement drive.

POST /api/admin/reminders/send — Manually trigger the daily email reminders.

GET /api/admin/schedules — Fetch all configured background job schedules.

POST /api/admin/schedules — Create a new background reminder/job schedule.

DELETE /api/admin/schedules/<id> — Delete an existing background job schedule.

GET, POST /api/company/drives — Fetch the company's drives or create a new placement drive.

GET /api/company/applications — Fetch all student applications for the company's drives.

POST /api/company/applications/<id>/status — Update a student's application status (e.g., Shortlisted, Selected).

GET /api/student/drives — Fetch approved placement drives that the student is eligible for.

GET, POST /api/student/profile — Fetch or update student profile details and resume.

POST /api/student/apply — Submit an application for a specific placement drive.

GET /api/student/history — Fetch the student's past application history.

POST /api/student/export — Trigger a background job to export application history to CSV.

GET /api/notifications — Fetch system notifications for the authenticated user.

POST /api/notifications/<id>/read — Mark a specific notification as read.

POST /api/test/daily-reminders — Trigger daily reminders manually via Celery (for testing).

---

7. Video Presentation

Drive Link:
https://drive.google.com/file/d/1oVLFFZDoYk0fOEN0yOI3AwkFmOtG_72O/view?usp=sharing

---